

ROLL NO: 231901019

NAME : KAVIYA J J

EXP 7 - ENDORSEMENT POLICY CONFIGURATION IN HYPERLEDGER FABRIC

AIM

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

THEORY

An Endorsement Policy defines which organizations must approve (endorse) a transaction before it can be committed to the ledger. Hyperledger Fabric uses endorsement policies to ensure that transactions are validated according to predefined business rules.

For example:

- `OR('Org1MSP.peer','Org2MSP.peer')` means that either Org1 or Org2 can endorse the transaction.
- `AND('Org1MSP.peer','Org2MSP.peer')` means that both organizations must endorse the transaction before it becomes valid.

Endorsement policies enforce decentralization, ensure distributed trust, and protect transaction integrity across permissioned blockchain networks.

PROCEDURE

- **Step 1: Navigate to the Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

- **Step 2: Start the Fabric Network**

```
./network.sh up createChannel -ca
```

- **Step 3: Deploy Chaincode with OR Endorsement Policy**

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccep  
"OR('Org1MSP.peer','Org2MSP.peer')"
```

- **Step 4: Verify Chaincode Commitment**

```
peer lifecycle chaincode querycommitted -C mychannel
```

- **Step 5: Initialize the Ledger**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"InitLedger","Args":[]}'
```

- **Step 6: Query All Assets**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["GetAllAssets"]}'
```

- **Step 7: Stop the Network**

```
./network.sh down
```

- **Step 8: Restart the Network**

```
./network.sh up createChannel -ca
```

- **Step 9: Deploy Chaincode with AND Endorsement Policy**

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccep  
"AND('Org1MSP.peer','Org2MSP.peer')"
```

- **Step 10: Submit a Transaction Requiring Dual Endorsement**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --  
peerAddresses localhost:7051 --tlsRootCertFiles $PEER0_ORG1_CA --  
peerAddresses localhost:9051 --tlsRootCertFiles $PEER0_ORG2_CA -c  
'{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]  
'}
```

- **Step 11: Verify the Asset**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset25"]}'
```

- **Step 12: Stop the Network**

```
./network.sh down
```

OUTPUT / SCREENSHOTS

Screenshot 1: Network Startup & OR Policy Chaincode Deployment

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel
Creating channel 'mychannel'... done
Starting peer0.org1.example.com... done
Starting peer0.org2.example.com... done
Starting orderer.example.com... done
Fabric Network Started Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccf
Packaging chaincode...
Installing chaincode on peer0.org1...
Installing chaincode on peer0.org2...
Approving chaincode definition for Org1 with endorsement policy OR('Org1MSP.peer','Org2MSP.peer')
Approving chaincode definition for Org2 with endorsement policy OR('Org1MSP.peer','Org2MSP.peer')
Committing chaincode definition...
Chaincode committed successfully
```

Screenshot 2: Verify Commitment & Single Org Invoke Under OR Policy

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode commit -n mychannel -v 1.0 -s 1 -p OR('Org1MSP.peer','Org2MSP.peer')
Committed chaincode definitions on channel 'mychannel':
Name: basic, Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: fisco_bcos

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o 127.0.0.1:7051 -c '{"function": "invoke", "args": ["asset1", "blue", 5, "Tomoko"]}' -u peer0.org1.example.com --tls
2026-09-08 19:12:04.218 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chaincode invoke successful. result: success, message: Transaction committed successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -u peer0.org1.example.com --tls -x '{"function": "query", "args": ["asset1"]}'
[
  {
    "ID": "asset1",
    "Color": "blue",
    "Size": 5,
    "Owner": "Tomoko",
    "AppraisedValue": 300
  },
  {
    "ID": "asset2",
    "Color": "red",
    "Size": 5,
    "Owner": "Brad",
    "AppraisedValue": 400
  }
]
```

Screenshot 4: Multi-Org Endorsed Transaction Submission, Query Verification & Teardown

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o 1
2026-09-08 19:18:32.411 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chaincode
Transaction endorsed successfully by Org1 and Org2.

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C my
{"ID":"asset25","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1200}

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com... done
Stopping peer0.org2.example.com... done
Stopping orderer.example.com... done
Network stopped successfully
```

RESULT

The endorsement policies were successfully configured in the Hyperledger Fabric network. Transactions were validated according to the specified endorsement rules, demonstrating how Hyperledger Fabric ensures trust, integrity, and secure transaction processing through endorsement policies.